

Multi-agent Reinforcement Learning-based In-place Scaling Engine for Edge-cloud Systems

Jovan Prodanov*, Blaž Bertalanic*, Carolina Fortuna*, Shih-Kai Chou*, Matjaž Branko Jurič†, Ramon Sanchez-Iborra‡ and Jernej Hribar*

*Jožef Stefan Institute, Ljubljana, Slovenia.

†Faculty of Computer and Information Science, University of Ljubljana, Ljubljana, Slovenia

‡Department of Information and Communication Engineering, University of Murcia, Murcia, Spain

Email: {blaz.bertalanic, carolina.fortuna, shih-kai.chou, jernej.hribar}@ijs.si

Abstract—Modern edge-cloud systems face challenges in efficiently scaling resources to handle dynamic and unpredictable workloads. Traditional scaling approaches typically rely on static thresholds and predefined rules, which are often inadequate for optimizing resource utilization and maintaining performance in distributed and dynamic environments. This inefficiency hinders the adaptability and performance required in edge-cloud infrastructures, which can only be achieved through the newly proposed in-place scaling. To address this problem, we propose the Multi-Agent Reinforcement Learning-based In-place Scaling Engine (MARLISE) that enables seamless, dynamic, reactive control with in-place resource scaling. We develop our solution using two Deep Reinforcement Learning algorithms: Deep Q-Network (DQN), and Proximal Policy Optimization (PPO). We analyze each version of the proposed MARLISE solution using dynamic workloads, demonstrating their ability to ensure low response times of microservices and scalability. Our results show that MARLISE-based approaches outperform heuristic method in managing resource elasticity while maintaining microservice response times and achieving higher resource efficiency.

Index Terms—Edge-cloud, Kubernetes, Auto-scaling, Multi-Agent Deep Reinforcement Learning, In-place scaling

I. INTRODUCTION

The cloud-native paradigm encompasses a set of principles and practices for designing, developing and managing software systems that fully leverage the capabilities of cloud computing [1]. It emphasizes key attributes such as scalability, performance and adaptability so that systems can dynamically adapt to varying loads. For example, a video streaming platform must scale its infrastructure to accommodate peak traffic during live events [2], ensuring uninterrupted viewing experiences for users. Similarly, a cloud service running a Machine Learning (ML) model might dynamically scale resources to make real-time predictions, such as identifying user location [3] and activity patterns in a given area based on the time of day, enabling targeted services or recommendations. Furthermore, as cloud infrastructures are connected to the edge, resulting in so-called edge-cloud systems [4], the infrastructure becomes more heterogeneous in terms of physical computing capability.

These scenarios, involving dynamic user demands and heterogeneous, distributed infrastructure, highlight the need for adaptive and efficient resource management solutions. As modern microservices typically follow a microservice architecture, resource scaling can be done at different levels,

such as the container, the pod, or the cluster [5]. For stateful microservices, i.e., microservices that maintain persistent data across requests, Vertical Pod Autoscaling (VPA) techniques that adjust the resource requests and limits (Central Processing Unit (CPU) and memory) are more suitable compared to Horizontal Pod Autoscaling (HPA), which adjusts the number of pods based on demand. However, existing scaling methods primarily rely on predefined rules or reactive thresholds, which lack context awareness and slow to adapt to dynamic and unpredictable loads [6]. Consequently, they struggle to handle fluctuations in request volume and state consistency requirements in stateful microservices. As a result, such solutions cannot optimize resource utilization in distributed environments, typically leading to under- or over-provisioning situations [5]. As also discussed by Coutinho *et al.* [7], achieving resource elasticity is a fundamental challenge in environments where loads are highly dynamic and resources are limited.

To enable resource allocation for stateful microservices, in-place scaling was recently proposed as a method to adjust CPU and memory resources without requiring pod restarts, allowing services to scale more efficiently while maintaining microservice state [8]. However, it has been noticed that there is a gap in the ability of existing VPA tools to minimize resource slack and respond promptly to throttling of stateful microservices, leading to increased costs and impacting crucial metrics such as throughput and availability [5]. Considering the dynamics of in-place scaling and the distributed, heterogeneous nature of edge-cloud VPA, a Multi-Agent Deep Reinforcement Learning (MADRL)-based solution [9], [10] that relies on distributed agents that learn and adapt through experience, seems particularly suitable for the design of a scalable autoscaling engine. Decentralized MADRL approaches offer a promising solution by exploiting the modular structure of cloud-native systems. Each agent can control the resource allocation for a given microservice in a cloud-based environment, resulting in an intelligent and scalable solution. In addition, MADRL enables collaboration between agents to maintain system-wide performance while meeting the dynamic demands of workloads, making it an ideal candidate to address resource elasticity management in cloud-native environments for microservices in-place scaling.

In this paper we make the following contributions:

- We propose a novel VPA solution for in-place scaling of stateful microservices, Multi-Agent Reinforcement Learning-based In-place Scaling Engine (MARLISE), and develop it using two well-known Deep Reinforcement Learning (DRL) algorithms: Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) to develop both discrete and continuous versions of the solution.
- In our experimental evaluation, we demonstrate that MARLISE can stably and dynamically scale resources while effectively reducing Key Performance Indicators (KPIs), such as response time, compared to a conventional heuristic baseline when the load varies dynamically over time, i.e., when the number of requests per microservice fluctuates.
- We show that the proposed solution can prioritize resources for specific microservices when necessary, whereas the heuristic approach is unable to do so.
- We also demonstrate that our solution is highly scalable and adapts seamlessly when stateful microservices are removed or added in the cloud.

The rest of the document is organized as follows: Section II provides a summary of related work, while Section III outlines the background and key challenges. Section IV introduces and elaborates on MARLISE, the proposed solution. Section V details the experimental evaluation methodology while Section VI discusses the results. Finally, Section VII concludes the paper.

II. RELATED WORK

Efficient resource management remains a significant challenge in edge-cloud systems, especially given their heterogeneous infrastructure and highly dynamic workloads. Traditional scaling methods rely heavily on static rules or thresholds, which fail to swiftly adapt to fluctuating demands [7]. To overcome these limitations, more advanced approaches utilizing ML, such as Convolutional Neural Network (CNN) [11] and Long Short-Term Memory (LSTM) [12], [13], have been proposed. Such approaches anticipate resource requirements based on historical data. However, these methods typically lack adaptability to sudden workload changes, which is critical in dynamic edge-cloud scenarios.

To that end, Reinforcement Learning (RL)-based approaches [14]–[22] have emerged as promising solutions for real-time resource allocation. For example, the works in [14], [15] integrate LSTM-based predictions with RL to enable more precise scaling decisions. The authors in [16] proposed a Deep Elastic Resource Provisioning (DERP) approach and demonstrated improvements in Virtual Machine (VM) scaling using DQN. Similarly, the Erlang autoscaler [17] employs a multi-armed bandit algorithm to allocate VMs to microservices, while in [22], RL was employed to allocate resources (both horizontally and vertically) in containerized cloud applications. Unfortunately, none of these approaches consider **in-place scaling** required for stateful services, nor do they account for distributed decision-making. Other related

TABLE I: Comparative Analysis of Scaling Approaches.

Approaches — Feature	Traditional ML [11]–[13]	RL-based [14]–[22]	MADRL [23]–[29]	Proposed MARLISE
Real-time adaptation	✗	✓	✓	✓
Distributed decision making	✗	✗	✓	✓
In-place scaling	✗	✗	✗	✓
Stateful microservice specific	✗	✗	✗	✓

efforts employed RL for task scheduling [18] and network management, such as slice admission control [19] or 5G radio access network slicing [20] and Quality of Service (QoS) optimization [21]. To address distributed decision-making requirements, MADRL approaches have emerged to handle decentralized control challenges across various domains, including resource allocation in cloud computing environments [23]–[25], network management in 5G and beyond [26]–[28], and wireless networks [29]. Nevertheless, existing MADRL methods have not yet been tailored to **in-place VPA** for stateful microservices, thus failing to resolve critical limitations around uninterrupted and fine-grained resource allocation.

Table I highlights how the proposed MARLISE method uniquely enables real-time, in-place scaling for stateful microservices compared to existing approaches. The proposed approach addresses identified gaps in the literature by employing MADRL specifically tailored for real-time in-place scaling, ensuring scalability, adaptability, and uninterrupted stateful microservice operations.

III. IN-PLACE AUTO-SCALING

In our work, we assume an edge-cloud environment realized through a cloud-native technology stack in which stateful applications realized as microservices are packaged as containers and managed by an orchestration platform. The most widely used and most versatile container orchestration solution is Kubernetes [30] and its flavours such as Microk8s [31].

A. Resource Allocation in Kubernetes Clusters

A Kubernetes cluster consists of a set of nodes that abstract the physical or virtual infrastructure and provide the environment for running containerized workloads. The cluster typically consists of worker nodes that host containerized applications and a master node (or control plane) that is responsible for managing the overall operation of the cluster, including orchestration and communication between the nodes. In this work, the term *microservices* refers specifically to applications encapsulated in containers, i.e., application code with its dependencies running in Kubernetes pods. Each node provides a finite pool of resources, such as CPU and memory, which is determined by the physical infrastructure of the host. These resources can be dynamically allocated to the microservices, as illustrated in Fig. 1.

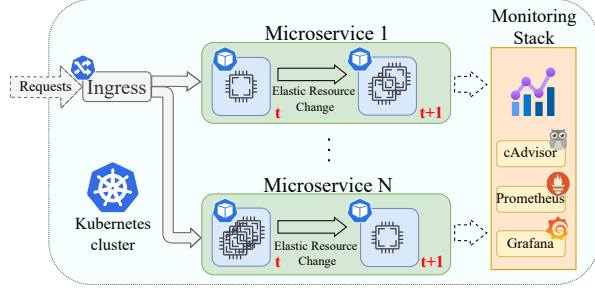


Fig. 1: Kubernetes cluster hosting N microservices running inside pods and dynamically adjusting resources over time. Between timestamps t and $t + 1$, microservice 1 increases its allocated resources, while microservice N decreases its resources.

In a Kubernetes environment, resources are shared between containers running on nodes and are initially managed by the Kubernetes scheduler, which allocates resources based on the specified requests and limits defined in the deployment configurations. Each container within a pod can request a certain amount of CPU and memory to ensure it has the minimum necessary to function, while limits define the maximum it can consume. When resource contention occurs, Kubernetes uses QoS classes to prioritize containers and ensures that higher priority microservices get the resources they need, while the remaining resources are distributed to lower priority microservices. After deployment, traditional scaling methods adjust resource specifications either horizontally through HPA, by adding or removing containers, or vertically through *traditional VPA*, by creating new containers with updated resources and removing the old containers. These approaches can lead to overhead, downtime, and inefficiencies. Nevertheless, the VPA in Kubernetes can dynamically adjust the requests and limits values according to a pluggable algorithm [5], so it is possible to create more intelligent and dynamic control logic. Furthermore, the newly proposed in-place resizing is able to dynamically adjust compute resources within a pod, without creating a new one and destroying the old one, a process that leads to interrupting microservices [8]. These two enablers pave the way to improved VPA techniques towards ensuring increased resource efficiency.

In order to ensure dynamic and intelligent *in-place VPA* scaling, the control logic typically relies on various metrics that can be collected to monitor resource usage and application performance, such as CPU and memory usage, disk I/O and network bandwidth. Tools such as Prometheus [32] and cAdvisor [33] can be integrated into the cloud-native technology stack to collect such data from the edge-cloud system at regular intervals. These metrics can be used to make informed scaling decisions and can be visualized and analyzed with Grafana [34].

B. Challenge of Dynamic Allocation of In-place Resources

In-place VPA resource allocation in edge-cloud environments presents a difficult problem: *How to allocate limited*

resources such as CPU and memory to each microservice in a way that maintains acceptable performance across the system without interruptions? To solve this problem we identify the following three challenges:

Adaptability to Dynamic Load Patterns: Effective *in-place VPA* scaling requires complex real-time decisions to dynamically adjust resource allocations to ensure minimal response times within a few seconds and prevent performance degradation in microservices. Traditional static heuristics or rule-based approaches struggle to cope with the unpredictability of load patterns. To enable adaptive decision-making in real time, advanced online learning techniques, such as classification models or RL, are essential. These methods enable the system to continuously learn from the evolving load and adapt resources autonomously.

Priority management: Ensuring that microservices with high-priority receive sufficient resources while preventing microservices with lower-priority from experiencing resource depletion and becoming unresponsive is a non-trivial task. Resource contention in constrained environments requires intelligent arbitration, not only deciding how much to allocate, but also determining which services should take priority when resources are scarce. This requires a sophisticated context-aware allocation mechanism that takes into account (i) the global resource constraints of the system, (ii) the diverse and dynamic requirements of individual microservices, and (iii) the need to maintain system-wide response time guarantees.

System scalability at maximum utilization: Microservices are elastic by nature, i.e., they can be dynamically instantiated or terminated as required. An *in-place VPA* scaling solution must therefore support the reallocation of resources for newly introduced microservices, even if all available resources are already allocated. This becomes particularly difficult when a new microservice requires additional resources and the system is forced to reallocate resources from currently running microservices without causing a cascading performance degradation. Any effective solution must have self-adaptive capabilities to ensure that scaling decisions remain efficient and balanced across the entire system.

Proposed MARLISE solution: To address these challenges in dynamic edge cloud environments, we adopt the MADRL framework, modeling each microservice as an independent learning agent that makes real-time scaling decisions. Unlike traditional heuristic-based or centralized methods, MADRL enables decentralized decision-making, reducing overhead and improving responsiveness. It continuously learns and adapts based on system feedback, effectively handling unpredictable loads. By integrating MADRL with in-place VPA scaling, we develop MARLISE, a self-optimizing, autonomous resource management system that enhances performance, scalability, and resource efficiency. The details of MARLISE are described in Section IV, while the next section examines the limitations of native auto-scaling solutions, providing the motivation for our proposed approach.

TABLE II: Comparison of native auto-scaling methods (Heuristic) with the proposed MARLISE method.

Scaling Feature/Scaling Approach	HPA	VPA	MARLISE
Time interval for scaling decision	15s	1m	1s
Best for stateless microservice	✓	✗	✗
Best for stateful microservices	✗	✓	✓
Support seamless scaling	✗	✗	✓
Ability to resize pods	✗	✓	✓
Adaptability to dynamic load patterns	✗	✗	✓
Priority management	✗	✗	✓
System scalability at maximum utilization	✗	✗	✓

C. Limitation of Native Auto-scaling Solutions

Kubernetes offers two native approaches to auto-scaling resources: HPA [35] and VPA [36]. HPA adjusts the number of pod replicas in a deployment based on observed CPU utilization or other selected metrics. VPA, still in the experimental phase of Kubernetes autoscaler functionality, allocates resources (e.g., CPU or memory) to the pod based on historical utilization. While HPA ensures that workloads can handle varying demand by scaling out or in, VPA optimizes resource allocation per pod to improve efficiency and reduce wasted capacity. However, both methods require restarting the container, resulting in significant overhead and adding a time delay in response. In other words, the native solutions lack support for seamless scaling, a key feature our proposed solution has.

In Table II, we outline the main differences between native solutions and the proposed MARLISE solution. Note that every approach supports both stateless and stateful microservices. However, MARLISE and VPA perform better for stateful services, while HPA is more effective for stateless services. Regarding the time interval for scaling decisions (i.e., the decision time-step), our proposed solution is able to make scaling decisions every second, whereas HPA scales every 15 seconds, and VPA scales pods approximately every minute. Furthermore, both MARLISE and VPA can resize pods by adjusting allocated resources, whereas HPA can only scale by adding additional pods, leading to coarser resource allocation. This makes VPA and MARLISE more granular in resource adjustments.

Finally, native solutions are not designed to address the challenges of in-place resource allocation, as discussed in the previous subsection. In contrast, our proposed solution is specifically tailored to overcome these limitations. While proposed MARLISE takes advantage of newly proposed in-place resizing feature in Kubernetes, it introduces a learned, dynamic logic to optimize resource allocation, making it highly adaptable to microservice which load patterns change in a matter of seconds. Additionally, it ensures prioritization, efficient resource management, and system scalability.

IV. MADRL FOR IN-PLACE VPA RESOURCE ALLOCATION

RL is, at its core, a sequential decision-making process in which agent learns to maximize cumulative rewards through interaction with the environment. This includes key concepts such as states, actions, and rewards, where the agent discovers a strategy (i.e., a set of actions) that maximizes the long-term

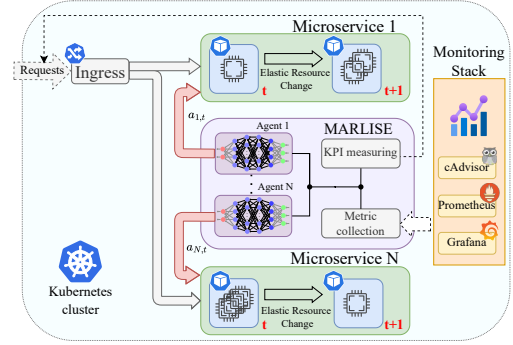


Fig. 2: Illustration of the proposed MARLISE setup showing how agents manage resource allocation for each microservice.

reward. In the case of MADRL, multiple agents work together to improve coordination and dynamically allocate resources, prioritize resource allocation, and enable a scalable approach to managing the system, such as dynamically adding and removing resources such as allocating CPUs or memory for a microservice. Unlike traditional methods, MADRL uses trial-and-error interactions with the environment to learn resource allocation in real time without the need for centralized control.

Dynamic workload management, a key challenge in such systems, requires real-time *scaling decisions* to minimize response times and avoid microservice degradation. This is enabled by learning techniques such as DRL, which are well suited for adaptive and real-time decisions. Similarly, priority management ensures that high-demand microservices receive the necessary resources without neglecting others. This involves not only deciding how to allocate resources, but also which microservices should take precedence when resources are limited in order to balance the varying demands of each microservice while maintaining system-wide performance metrics such as response time. In addition, the scalability of the system requires the ability to dynamically add or remove microservices. MADRL addresses these challenges (see Section III-B) by allowing agents to work independently and coordinate to effectively achieve system-level goals.

Each agent in the proposed framework monitors and allocates resources to a specific microservice within the system, as shown in Fig. 2. The main goal of each agent is to dynamically in-place adjust the resource allocation for the assigned microservice, ensuring that KPIs such as response time and throughput are met while staying within predefined resource utilization thresholds. This decentralized approach increases scalability and flexibility as agents can manage their microservices independently while optimizing the system together. MADRL also enhances robustness, as the suboptimal behavior of one agent can be compensated by others, ensuring system-wide performance under varying conditions.

We propose *MARLISE-Discrete* and *MARLISE-Continuous*, each tailored to the specific challenges of dynamic resource allocation, priority management and scalability in edge cloud environments, while enabling adaptive and efficient decision making across the system.

A. States, Action, and Reward signal

To ensure that the agents have sufficient information regarding the system, we design the **state** to encompass seven variables that describe the current snapshot of the system:

- Resource Limit - The current allocation of resources;
- Resource Usage - Consumption, the gap between the limits and the consumption can indicate whether an increase or decrease is required;
- Available Resources - Availability, the agent is aware of the system's free resources;
- Utilization - Efficiency of the allocated resources;
- Average utilization of other resources - Knowledge about the resource utilization of other agents;
- Priority - Indicator of how important the microservice is;
- Average priority of other microservices - Helps the system to make trade-offs by taking into account the relative importance of all running services.

In addition, a time window of past variables is maintained, with each variable containing multiple past values stored in a First-In, First-Out (FIFO) order. This approach enables the model to leverage historical context for each variable, facilitating more informed decision-making based on recent system behavior. Specifically, the state space consists of $7 \times k$ input neurons, where k represents the number of past values stored for each variable. To enhance learning, state values are normalized, which enables relative resource allocation and consistency across different resource levels. This normalization improves the efficiency of DRL models by standardizing inputs and helping the model to focus on meaningful patterns and generalize optimal allocation strategies.

Actions are divided into two types: Discrete and continuous. MARLISE-Discrete operates in a discrete action space and selects from a set of predefined actions, such as increasing, decreasing or maintaining resource levels — ideal for straightforward, categorical adjustments. In contrast, MARLISE-Continuous use a continuous action space and generate actions as float values in the range $[-1, 1]$. These values are scaled and applied as precise adjustments to resource allocations, allowing for granular and flexible control.

The **reward signal** balances two objectives: encouraging efficient CPU utilization within desirable thresholds, and minimizing response times, particularly for high-priority microservices. Let $\eta_i(t)$ represent the utilization of the i -th microservice at time step t , with upper and lower utilization thresholds denoted as η_U and η_L respectively. The utilization reward $\rho_i(t)$ for each agent, which measures how efficiently the resources are being used at time step t and is defined as:

$$\rho_i(t) = \begin{cases} 1 + \frac{\eta_i(t)}{\eta_U - \eta_L} & \text{if } \eta_L \leq \eta_i(t) \leq \eta_U \\ \min\left(\frac{\Delta_i(t)}{10}, 1\right) & \text{if } \eta_i(t) < \eta_L \text{ and } \Delta_i(t) > 0 \\ \max\left(\frac{\Delta_i(t)}{10}, -1\right) & \text{if } \eta_i(t) < \eta_L \text{ and } \Delta_i(t) \leq 0 \\ 0 & \text{Otherwise} \end{cases}, \quad (1)$$

where $\Delta_i(t) = \eta_i(t) - \eta_i(t-1)$, represents the change in utilization compared to the previous timestep. The Eq. 1 dis-

courages both low and excessive utilization while incentivizing higher utilization towards the upper threshold η_U .

Let $\omega(t)$ denote the weighted response time at time step t , which contains the priority p_i and the response times σ_i of each service:

$$\omega(t) = \sum_{i=1}^N (1 + p_i) \cdot \sigma_i(t). \quad (2)$$

The weighted response time as defined in Eq. 2 aggregates response times across all microservices, emphasizing those with higher priority. This ensures critical microservices have greater influence on the overall reward, promoting prioritization during resource allocation.

A **shared reward** $r_s(t)$ is then defined to minimize the overall response times in the system and act as a centralized reward signal for collaboration between all agents:

$$r_s(t) = 1 - \alpha \cdot (\omega(t) - 0.01), \quad (3)$$

with α denotes the scaling factor assigned to response time. A higher α places greater emphasis on overall system performance relative to individual agent performance. According to above equation, $r_s(t)$ provides a positive reward when the response time approaches zero. This shared reward encourages the agents to work together towards the system-wide goal of reducing response times.

The **reward** signal $r(t)$ combines individual utilization efficiency with the overall response time performance, ensuring a balance between resource usage and microservice response time:

$$r_i(t) = \beta \cdot \rho_i(t) + r_s(t). \quad (4)$$

The scaling factor $\beta \in (0, 1]$ captures how important agent utilization is in the reward signal. Note that the reward is determined after every agent in the system has selected an action and the system has transitioned to the next state.

B. Multi-agent solutions

1) **MARLISE Discrete**: Is based on the DQN approach [37]. In the MARLISE-Discrete algorithm, each agent learns to approximate a Q-value function $Q(s, a; \theta)$ using a deep neural network [37], where s represents the current state, a the selected action and θ the parameters of the neural network. Through repeated interactions with the environment, the agents learn the Q-values associated with each action-pair in such a way to be able to select the action with the highest expected reward in each state. For example, the agents add or remove a predetermined amount of CPU and memory to individual microservices. If the response time is high, resources are allocated to the microservice, while resources are removed if utilization is low.

As outlined in Alg. 1, each agent independently maintains a Q-network Q_i , a target network $\hat{Q}_i$, and an experience buffer $\mathcal{D}_i$. The agents store transition $(s_i(t), a_i(t), r_i(t), s_i(t+1))$, i.e., experience, in the $\mathcal{D}_i$ buffer and then sample mini-batches of size J to update their Q-networks when the experience buffer $\mathcal{D}_i$ reaches the defined size. The target network provides

Algorithm 1 Proposed MARLISE-Discrete algorithm

```

1: for  $i = 1$  to  $N$  (agents) do
2:   Initialize experience buffer  $\mathcal{D}_i$ , Q-network  $Q_i(s, a; \theta_i)$ 
   with random weights, and target Q-network
    $\hat{Q}_i(s, a; \theta_i^-)$  with weights  $\theta_i^- = \theta_i$ 
3: end for
4: for  $k = 1$  to  $K$  (episodes) do
5:   for  $t = 1$  to  $T$  (timesteps) do
6:     for  $i = 1$  to  $N$  (agents) do
7:       Select action  $a_i(t)$  using  $\epsilon$ -greedy policy
8:       Execute action  $a_i(t)$ 
9:       Observe the new state  $s_i(t+1)$  and determine the
       reward  $r_i(t)$  according to Eq. 4
10:      Save transition  $(s_i(t), a_i(t), r_i(t), s_i(t+1))$ 
11:      Randomly sample  $J$  experiences from  $\mathcal{D}_i$ 
12:      for every  $\{s_i(j), r_i(j), a_i(j), s_i(j+1)\}$  in batch do
13:        Set  $y_i(j) = r_i(j) +$ 
         $\gamma \max_{a_i} \hat{Q}_i(s_i(j+1), a_i(j+1))$ 
14:      end for
15:      Calculate loss:
         $\mathcal{Z}_i = \frac{1}{J} \sum_{j=1}^J (Q_i(s_i(j), a_i(j)) - y_i(j))^2$ 
16:      Update  $Q_i(s_i, a_i | \theta_i)$  by minimising the loss  $\mathcal{Z}_i$ 
17:      Update target network:
         $\theta_i^- \leftarrow \tau \theta_i + (1 - \tau) \theta_i^-$ 
18:    end for
19:  end for
20: end for

```

stable Q-value estimates using the Bellman equation (line 13). The Q-network is then trained by minimizing the loss between the current and target Q-values (line 16), while a soft update of the target network (line 17) ensures stable learning and reduces the risk of policy divergence.

2) *MARLISE Continuous*: Is based on the PPO approach [38]. Each agent employs an actor-critic architecture, where the actor network selects actions and the critic network estimates state values to drive policy updates. The solution relies on a stochastic policy in continuous action spaces that allows agents to adaptively select actions based on probabilistic optimization. Each agent's actor network π_i is individually updated by policy gradients to improve adaptability to changing states by performing gradient descent on Eq. 5. Note that resource allocation with MARLISE-Continuous adjustments are granular and occur within a predefined range.

MARLISE-Continuous uses the vanilla advantage estimation method (line 14 in Alg. 2) to compute the advantage function $\hat{A}_i$ for agent i , which measures how much better (or worse) performing a particular action a_i in a state s_i is compared to the average action in that state according to the current policy. The most important equation in the Multi-Agent PPO (MA-PPO) algorithm is the clipped surrogate loss in Eq. 5, which ensures that the new policy does not deviate significantly from the old one. This approach helps to maintain the stability of the training and avoids large, destructive updates. Let $r_i(\theta) = \frac{\pi_{\theta_i}(a_i | s_i)}{\pi_{\theta_i^{\text{old}}}(a_i | s_i)}$ is the ratio of the probability that the agent i takes the action a_i in state s_i in

Algorithm 2 Proposed MARLISE-Continuous algorithm

```

1: for  $i = 1$  to  $N$  (agents) do
2:   Initialize actor network  $\pi_{\theta_i}(a | s)$ , critic network  $V_{\phi_i}(s)$ 
   with weights  $\phi_i$  and buffer  $\mathcal{D}_i$ 
3: end for
4: for  $m = 1$  to  $M$  (episodes) do
5:   for  $t = 1$  to  $T$  (timesteps) do
6:     for  $i = 1$  to  $N$  (agents) do
7:       Sample action  $a_i(t) \sim \pi_{\theta_i}(a | s_i(t))$ 
8:       Execute action  $a_i(t)$ 
9:       Observe the new state  $s_{i,t+1}$  and determine the
       reward  $r_i(t)$  according to Eq. 4
10:      Store transition  $(s_i(t), a_i(t), r_i(t), s_i(t+1))$  in  $\mathcal{D}_i$ 
11:    end for
12:  end for
13:  for  $i = 1$  to  $N$  (agents) do
14:    Compute  $\hat{A}_i = R_i - V_{\phi_i}(s_i)$  for transitions in  $\mathcal{D}_i$ 
15:    Compute  $\mathcal{L}_i^{\text{clip}}(\theta_i)$  for transitions  $\mathcal{D}_i$  as in Eq. 5
16:    for  $k = 1$  to  $K$  (epochs) do
17:      Update  $\theta_i$  by maximizing  $\mathcal{L}_i^{\text{clip}}(\theta_i)$ 
18:      Update  $\phi_i$  by minimizing the value loss:
         $L_V(\phi_i) = \mathbb{E}[(V_{\phi_i}(s_i) - R_i)^2]$ 
19:    end for
20:    Update  $\theta_i^{\text{old}} \leftarrow \theta_i$  and clear buffer  $\mathcal{D}_i$ 
21:  end for
22: end for

```

the new policy relative to the old policy. This ratio effectively measures the divergence between the policies. The **clipped surrogate loss** $\mathcal{L}_i^{\text{clip}}(\theta_i)$ for agent i is then defined as:

$$\mathbb{E} \left[\min \left(r_i(\theta) \hat{A}_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right]. \quad (5)$$

The clipping mechanism restricts the policy update by limiting the ratio $r_i(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$. This ensures that the updates do not push the new policy too far away from the old policy, reducing the risk of destabilizing the training process. Unlike the other proposed MARLISE methods, MARLISE-Continuous discards past experience after each policy update and focuses solely on the most recent data to improve training relevance and responsiveness to current conditions in line 20.

V. EVALUATION METHODOLOGY

In this section, we describe the edge-cloud infrastructure, containerized microservice deployment, training process, and evaluation KPIs, followed by the heuristic baseline for comparison, which we use in the next evaluation section.

A. Implementation and Realistic Deployment Considerations

Dynamic in-place scaling is enabled by employing resource resizing feature of Kubernetes [8], allowing seamless CPU and memory adjustments without container restarts. Communication with the Kubernetes cluster is implemented via the official Kubernetes Python client [39], ensuring standard-compliant real-time resource adjustments. Real-time system metrics such as CPU utilization, allocated resources, and available resources

TABLE III: Microservice load levels during evaluation.

Time (s) / Microservice	1	2	3
0-7	25.0%	8.5%	66.5%
8-14	25.0%	72.0%	3.0%
15-21	47.0%	7.0%	46.0%

are collected every second using cAdvisor, integrated with Prometheus for data collection, and visualized using Grafana, accurately replicating realistic operational monitoring. Furthermore, the source code of the proposed MARLISE solution is publicly available for validation and replication purposes¹.

Our experimental evaluation is conducted on an edge-cloud cluster comprising two Raspberry Pi 5 nodes (64-bit, 2.4 GHz quad-core ARM CPUs) acting as worker nodes, and one VM equipped with a quad-core Xeon E5-2650 CPU at 2.00 GHz serving as the Kubernetes master node. Microk8s [31] orchestrates the environment due to its lightweight and ARM compatibility, suitable for realistic edge-cloud deployments.

B. Test Stateful Microservice and Workload Scenario

The evaluation utilizes **Localization as a Service (LaaS)**, a representative ML-driven microservice for device localization based on Bluetooth Low Energy (BLE) beacons. LaaS employs a fingerprinting method leveraging a pre-trained localization model to perform real-time inference, estimating device positions within a predefined grid. For realistic deployment, the LaaS microservice is containerized using Docker [40] and deployed within the Kubernetes environment, as demonstrated in our prior work [41].

The microservice predominantly consumes CPU resources, maintaining relatively constant memory utilization; thus, the evaluation explicitly focuses on CPU resource allocation under dynamically varying workloads that emulate realistic Internet of Things (IoT) or smart-city scenarios, characterized by significant fluctuations in user demand patterns. More specifically, during the evaluation of dynamic load patterns (Section VI-A) and priority management (Section VI-B), the system receives 100 requests per second, distributed as shown in Table III. Each request uses a LaaS microservice to determine the location of the device. During the scalability evaluation (Section VI-C), Table IV lists the number of requests received by each microservice.

C. MARLISE Training

The **training** of the MARLISE solutions was performed with a synthetic load at random intervals and with an intensity limited by an upper bound. This approach mimics the variability of real-world traffic (or load) while ensuring that the system operates within feasible resource limits so that agents can adapt effectively under dynamic conditions. Note that we use these pre-trained models in our evaluation.

Table V lists the fine-tuned parameters we use for the MARLISE solutions. Each exploration rate, i.e., ϵ -decay, OU Noise and action variance, is set to decrease gradually. In

¹The code supporting our experiments is publicly available on GitHub: <https://github.com/sensorlab/agent-edge-autoscaling>

TABLE IV: Request distribution for scalability evaluation.

Time (s) / Microservice	1	2	3	4
0-15	30	40	—	—
16-30	30	40	50	—
31-45	30	40	50	50
46-60	—	40	50	50

TABLE V: MARLISE parameters.

Parameter/Algorithm	Discrete	Continuous
Learning rate	1×10^{-4}	—
Actor learning rate	—	3×10^{-4}
Critic learning rate	—	1×10^{-3}
Discount factor γ	0.99	0.99
Replay buffer size	1000	—
Batch J size	128	600
Soft update parameter τ	0.005	—
Exploration rate	ϵ decay	Variance
Clip range	—	0.2
Entropy coefficient	—	0.01
Update epochs (K)	—	10
Num. of hidden layers	3	4
Neurons per hid. layer	64 – 128 – 64	64 – 128 – 128 – 64
Upper util. threshold η_U	60%	60%
Lower util. threshold η_L	30%	30%
Shared reward factor α	5	5
Util. reward factor β	0.5	0.5

this way, exploration is encouraged at first and after the first few training episodes, the system moves on to exploitation. The learning rates, the size of the replay buffers and the batch sizes have been optimized for stability and efficiency of performance. The update epochs for *MARLISE-Continuous* ($K = 10$) were selected to maintain a balance between training stability and computational efficiency. The reward function scaling factors, α and β , defined in Eq. 3 and Eq. 4, along with the threshold parameters η_L and η_U (Eq. 1), were fine-tuned for the target microservice and deployment setting.

D. KPIs

The KPIs evaluated included mean response time (the time it takes the cluster to process a request), violation rate (response times > 250 ms), resource utilization (percentage of allocated resources used) and resource deltas, which represent the total resource changes. To obtain accurate and stable results for the evaluation experiments, the MADRL algorithms were executed and their results averaged over 20 iterations.

E. Baseline for Comparison

As we highlighted in Section III-C, native HPA and VPA are not designed for uninterrupted scaling. Therefore, a direct comparison with our method would be inherently unfair, as they would perform worse in every experiment. Furthermore, our proposed solution dynamically adapts to request fluctuations within seconds, a responsiveness that the native approaches lack. To ensure a fair evaluation, we compare our method to a **heuristic** approach that was developed based on the guidelines in [42]. It uses a policy-driven technique that adjusts resource allocation based on CPU and memory usage trends and employs predefined thresholds to trigger scaling actions. This ensures dynamic resource adjustments

while maintaining system stability. The method continuously monitors utilization to prevent under-allocation (which leads to performance degradation) and over-allocation (which is inefficient). Scaling decisions are made every second to ensure real-time responsiveness and a fair comparison with the proposed solution. These threshold-based scaling approaches, as used in Kubernetes' HPA and VPA, are computationally efficient but struggle with unpredictable load patterns in dynamic multi-service environments, as we show in the next section.

VI. EVALUATION

In this section, we evaluate the proposed MARLISE solution described in Section IV, according to the methodology in Section V, demonstrating how it overcomes the challenges identified in Section III-B.

A. Evaluating Adaptability to Dynamic Load Patterns

In the first experiment, we evaluate how effectively the proposed solutions and the heuristic baseline respond to dynamic changes in load, i.e., addressing the first challenge of in-place scaling described in Section III-B. To collect performance data, the system is subjected to a workload of parallel requests per second, simulating user interactions with three LaaSs. Fig. 3 shows the performance of the heuristic approach and the MARLISE algorithms under two load changes between microservices, as described in Table III. The figures present the response time σ_i of each service i and CPU utilization over time and illustrate the effectiveness of each algorithm in allocating resources for the respective LaaS, which mainly relies on CPU resources for operation.

The results shown in Fig. 3 illustrate how the different algorithms react to dynamic changes in the number of requests to individual services. As can be seen from the response time graphs, each solution is able to adapt to changes within a few seconds. Among them, MARLISE-Continuous has the fastest response time compared to MARLISE-Discrete and the heuristic approach. A more detailed performance evaluation can be found in Table VI, which contains the averaged results over 20 experimental iterations. These results include the KPIs described in Section V-D and show significant performance differences between the algorithms. For example, in terms of violations and mean response time for microservice 3, the proposed Continuous solution shows superior performance, achieving a 0.14s response time and the lowest violations at 12.05%. In contrast, Discrete and heuristic achieve much higher response times (0.31s and 0.28s) and violations (24.53% and 25.99%), respectively. However, when considering the total amount of resources allocated to microservice 3 throughout the experiment, the baseline heuristic allocates the least resources (578mc), while Discrete allocates 648mc and Continuous allocates 667mc. This illustrates a fundamental trade-off between speed and resource efficiency: both MARLISE-Discrete and even more MARLISE-Continuous prioritize fast response times, but do so at the cost of more frequent resource allocation. This indicates that the heuristic approach is more conservative in

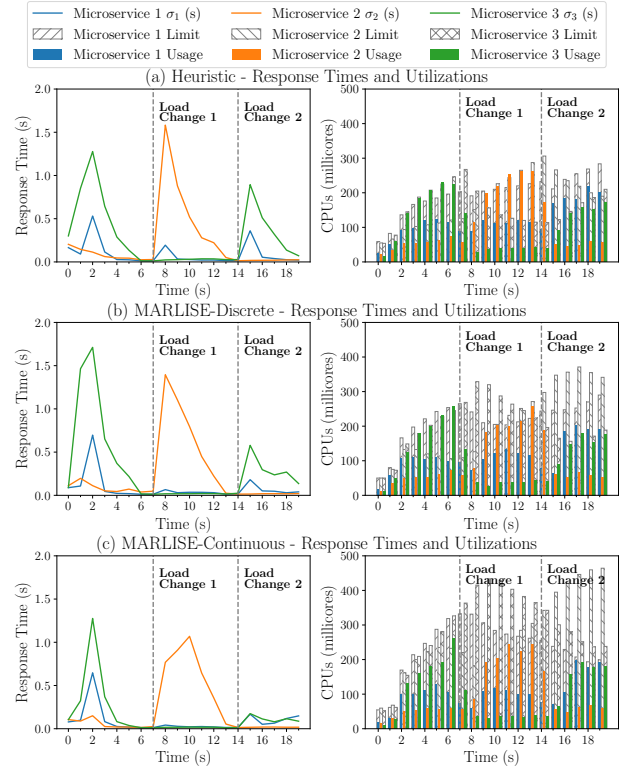


Fig. 3: Averaged performance of different algorithms under dynamic load changes.

resource allocation, resulting in higher response times and more violations.

B. Evaluating Priority Management

In our second experiment, we focus on determining how well the proposed solutions can adapt to priority management, which corresponds to the second challenge outlined in Section III-B. To evaluate the priority management, the system was subjected to the microservice load distribution described in Table III, with priority values p_i assigned to the agents depending on the importance of each microservice. The reward signal defined in Eq. 2 encourages resource allocation according with these priorities.

Table VII displays performance metrics for the KPIs listed in Section V-D in different combinations of priorities of their microservices. In Table VII (a), the priorities set in the MARLISE algorithms ensure that Microservice 2, which has the highest priority, consistently receives the highest share of CPU resources to maintain low response times even when the load changes. Microservice 1, which has the lowest priority, receives only minimal resources, which leads to slower response times under heavy load. However, MARLISE-Discrete allocates resources effectively and achieves the best response times and the fewest violations. In addition, Table VII (b) illustrates similar performance when a different microservice priority is high. The results also show that the heuristic method has the highest percentage of violations, while the

TABLE VI: Averaged performance (20 iterations) metrics for dynamic load experiment.

Metrics/Algorithm Microservices	Heuristic			MARLISE-Discrete			MARLISE-Continuous		
	1	2	3	1	2	3	1	2	3
Violations (%)	10.43	19.60	25.99	7.14	21.12	24.53	8.67	19.26	12.05
Mean response time (s)	0.09	0.22	0.28	0.08	0.24	0.31	0.08	0.22	0.14
Mean resource delta (mc)	415	488	578	645	615	648	532	709	667

TABLE VII: Averaged performance (20 iterations) metrics for different priority experiment.

(a) Low High Medium.

Metrics/Algorithm Microservices	Heuristic			MARLISE-Discrete			MARLISE-Continuous		
	1 (L)	2 (H)	3 (M)	1 (L)	2 (H)	3 (M)	1 (L)	2 (H)	3 (M)
Violations (%)	10.43	19.60	25.99	8.45	19.63	24.63	12.18	17.89	21.06
Mean response time (s)	0.09	0.22	0.28	0.09	0.21	0.30	0.14	0.29	0.27
Mean resource delta (mc)	415	488	578	264	475	598	359	574	557

(b) Medium Low High.

Metrics/Algorithm Microservices	Heuristic			MARLISE-Discrete			MARLISE-Continuous		
	1 (M)	2 (L)	3 (H)	1 (M)	2 (L)	3 (H)	1 (M)	2 (L)	3 (H)
Violations (%)	10.43	19.60	25.99	7.47	19.56	23.46	9.53	20.87	16.81
Mean response time (s)	0.09	0.22	0.28	0.08	0.24	0.26	0.11	0.34	0.21
Mean resource delta (mc)	415	488	578	356	366	664	374	499	542

(c) High Medium Low.

Metrics/Algorithm Microservices	Heuristic			MARLISE-Discrete			MARLISE-Continuous		
	1 (H)	2 (M)	3 (L)	1 (H)	2 (M)	3 (L)	1 (H)	2 (M)	3 (L)
Violations (%)	10.43	19.60	25.99	7.10	20.11	18.66	8.17	20.29	30.50
Mean response time (s)	0.09	0.22	0.28	0.08	0.22	0.26	0.09	0.30	0.38
Mean resource delta (mc)	415	488	578	504	399	444	413	571	550

TABLE VIII: Averaged performance (20 iterations) metrics for the scalability experiment.

(a) Agent 3 added.

Metrics/Algo. Microservices	Heuristic				MARLISE-Discrete				MARLISE-Continuous			
	1	2	3	4	1	2	3	4	1	2	3	4
Violations (%)	0.00	0.00	23.40	—	0.00	0.00	17.03	—	0.00	0.00	22.21	—
Mean response time (s)	0.02	0.02	0.27	—	0.02	0.02	0.26	—	0.02	0.02	0.37	—
Mean resource delta (mc)	29	44	305	—	450	475	370	—	102	96	237	—

(b) Agent 4 added.

Metrics/Algo. Microservices	Heuristic				MARLISE-Discrete				MARLISE-Continuous			
	1	2	3	4	1	2	3	4	1	2	3	4
Violations (%)	0.00	0.00	0.32	81.90	0.00	0.00	0.00	45.99	0.00	0.00	0.00	81.86
Mean response time (s)	0.02	0.02	0.02	1.69	0.02	0.02	0.02	0.71	0.02	0.02	0.02	1.42
Mean resource delta (mc)	2	1	6	7	131	244	262	139	147	171	176	66

(c) Agent 1 removed.

Metrics/Algo. Microservices	Heuristic				MARLISE-Discrete				MARLISE-Continuous			
	1	2	3	4	1	2	3	4	1	2	3	4
Violations (%)	—	0.00	0.00	18.67	—	0.00	0.00	0.00	—	0.00	0.00	11.11
Mean response time (s)	—	0.02	0.02	0.21	—	0.02	0.02	0.03	—	0.02	0.02	0.09
Mean resource delta (mc)	—	5	42	195	—	99	143	115	—	35	38	170

MARLISE solutions optimize the system-wide metrics even when complying to the priority settings.

Table VII shows that MARLISE-Discrete and MARLISE-Continuous best maintain the priority settings and effectively balance the KPIs. Our proposed solutions also allocate resources according to the set importance of the microservice,

thus successfully balancing response times and resource utilization. Additionally, the results indicate that the heuristic method only serves as a useful baseline, as it does not respond to priority changes. These results suggest that MARLISE-Discrete and MARLISE-Continuous are much better suited for environments that require precise prioritization and adaptive

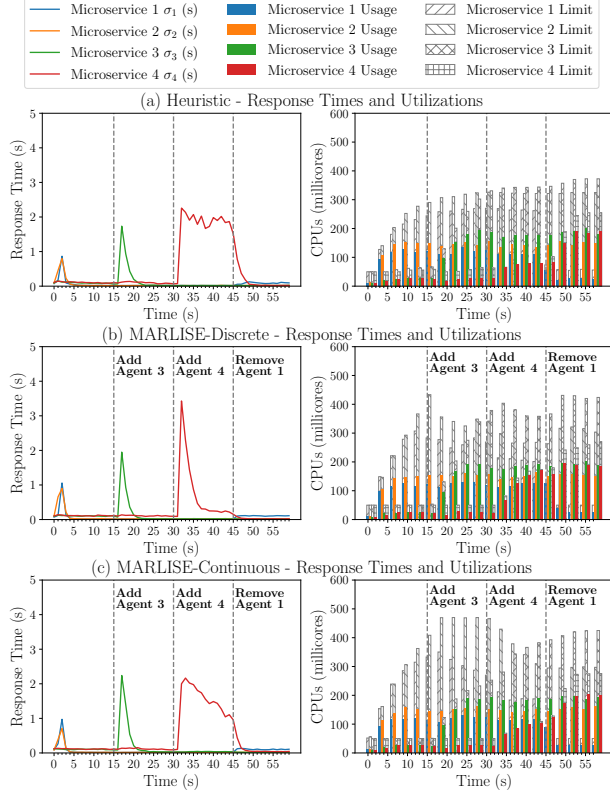


Fig. 4: Scalability experiment with dynamic addition and removal of agents.

scaling, and highlight the strengths of employing DRL in managing complex, priority-driven workloads.

C. Evaluating system scalability at maximum utilization

In our last experiment, we evaluate the scalability, i.e., the ability to add and remove agents, of the proposed in-place scaling solution. The experiment starts with two agents, adds a third agent after 15 seconds and introduces a fourth agent 15 seconds later, following the microservice load distribution outlined in Table IV. Finally, the first agent is removed to simulate a Kubernetes environment where microservices change frequently. This experiment tests the ability of each algorithm to manage limited resources under dynamic conditions and evaluates their responsiveness to changes in demand and agent configuration, even when resources are already fully allocated. Fig. 4 shows how each algorithm responds to these changes in real time.

At the start of the experiment, every approach handles the workload well. However, the heuristic approach, as shown in Fig. 4(a), has significant problems when agent 4, i.e. the fourth microservice, is added, as the response times become extremely high, indicating the limited scalability of the heuristic approach when the existing resources are already allocated and additional microservices are added. In contrast, MARLISE solution, as shown in Fig. 4(b) and Fig. 4(c) respectively, demonstrate significant better performance. Proposed solutions

efficiently allocate resources to the additional agents and stabilize quickly after the initial peaks. This is particularly noticeable in the response times for Microservice 4, while the CPU utilization for all microservices remains balanced.

Such a behavior is also confirmed in Table VIII, which shows the average performance over 20 iterations for metrics for every solution. The results also show that MARLISE-Discrete achieves the best results for violations, response times and resource deltas. These deltas indicate efficient collaboration between agents as resources are dynamically added or removed to optimize allocation without over provisioning. In each interval, MARLISE-Discrete stands out for its responsiveness to scaling changes, closely followed by MARLISE-Continuous, both of which exhibit adaptive and stable performance.

The scalability tests show that MARLISE is an effective algorithm for managing dynamic edge-cloud environments with microservice changes under limited resources. MARLISE-Discrete is characterized by maintaining low response times through fast adaptation, which makes it ideal for stable loads, while the adaptability of MARLISE-Continuous is beneficial in fluctuating conditions. The heuristic approach that served as the baseline, lacks the adaptability required for informed scaling decisions. These results highlight the potential of MARLISE solution for scalable, adaptive resource management in edge cloud systems.

VII. CONCLUSION

In this paper, we proposed a MADRL-based in-place VPA scaling engine and developed two scaling solutions based on DRL algorithms: MARLISE-Discrete and MARLISE-Continuous. We evaluated the proposed solutions in three scenarios: Adaptability to dynamic load changes, priority management, and scaling the system when system resources are fully utilized. Our results show the effectiveness of the proposed solutions in managing dynamic workloads. Moreover, our results have shown that MARLISE-Continuous ensures stable resource allocation by avoiding overutilization. For priority allocations, both MARLISE-Discrete and MARLISE-Continuous effectively satisfied the KPIs, with MARLISE-Discrete exhibiting the best scalability, closely followed by MARLISE-Continuous. Overall, the MARLISE solution outperformed the heuristic method by retaining the set KPIs and improving system performance. Future work will focus on extending DRL to manage additional edge cloud scenarios, considering ML scheduling techniques, and refining MARLISE for seamless integration and universal deployment.

ACKNOWLEDGEMENTS

This work was funded in part by the Slovenian Research Agency (grants P2-0016 and MN-0009-106), by grant PID2023-148104OB-C43 funded by MICIU/AEI/10.13039/501100011033 and co-funded by ERDF/EU, by the European Commission NANCY project (No. 101096456), and by the HORIZON-MSCA-PF project TimeSmart (No. 101063721).

REFERENCES

- [1] N. Kratzke and P. C. Quint, "Understanding cloud-native applications after 10 years of cloud computing - A systematic mapping study," *Journal of Systems and Software*, vol. 126, pp. 1–16, 2017.
- [2] H. Wang, Z. Long, H. Dong, and A. El Saddik, "MADRL-Based Rate Adaptation for 360° Video Streaming With Multiviewpoint Prediction," *IEEE Internet Things J.*, vol. 11, no. 15, pp. 26 503–26 517, 2024.
- [3] N. Garg and N. Roy, "Sirius: A Self-Localization System for Resource-Constrained IoT Sensors," in *Proc. MobiSys 2023*. New York, NY, USA: Association for Computing Machinery, 2023, p. 289–302.
- [4] P. Souza, T. Ferreto, and R. Calheiros, "Maintenance Operations on Cloud, Edge, and IoT Environments: Taxonomy, Survey, and Research Challenges," *ACM Comput. Surv.*, vol. 56, no. 10, Jun. 2024.
- [5] A. Pavlenko, J. Cahoon, Y. Zhu, B. Kroth, M. Nelson, A. Carter, D. Liao, T. Wright, J. Camacho-Rodríguez, and K. Saur, "Vertically Autoscaling Monolithic Applications with CaaS: Scalable Container-as-a-Service Performance Enhanced Resizing Algorithm for the Cloud," in *Proc. SIGMOD/PODS 2024*. New York, NY, USA: Association for Computing Machinery, 2024, p. 241–254.
- [6] A. Rubak and J. Taheri, "Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms," in *Proc. IEEE/ACM 16th International Conference on Utility and Cloud Computing*, ser. UCC '23. New York, NY, USA: Association for Computing Machinery, 2024.
- [7] E. F. Coutinho, F. R. de Carvalho Sousa, P. A. L. Rego, D. G. Gomes, and J. N. de Souza, "Elasticity in cloud computing: a survey," *Annals of Telecommunications*, vol. 70, no. 7, pp. 289–309, 2015.
- [8] Kubernetes, "Kubernetes Documentation: Resize a Container's Resources," 2023, accessed: 2024-11-28. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources>
- [9] W. Du and S. Ding, "A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications," *Artificial Intelligence Review*, vol. 54, no. 5, pp. 3215–3238, 2021.
- [10] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep Reinforcement Learning for Multiagent Systems: A Review of Challenges, Solutions, and Applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, 2020.
- [11] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: ML-based and QoS-aware resource management for cloud microservices," in *Proc. 26th ACM Int. Conf. ASPLOS*. New York, NY, USA: Association for Computing Machinery, 2021, pp. 167–181.
- [12] N.-M. Dang-Quang and M. Yoo, "Deep Learning-Based Autoscaling Using Bidirectional Long Short-Term Memory for Kubernetes," *Applied Sciences*, vol. 11, no. 9, 2021.
- [13] M. P. Yadav, Rohit, and D. K. Yadav, "Resource Provisioning Through Machine Learning in Cloud Services," *Arabian Journal for Science and Engineering*, vol. 47, no. 2, pp. 1483–1505, 2022.
- [14] P. Liu, W. Zhao, B. Zhang, and J. Wang, "Hybrid Elastic Scaling Strategy for Container Cloud based on Load Prediction and Reinforcement Learning," *Journal of Physics: Conference Series*, vol. 2732, p. 012014, 2024.
- [15] F. Lotfi and F. Afghah, "Open RAN LSTM Traffic Prediction and Slice Management Using Deep Reinforcement Learning," *57th Asilomar Conference on Signals, Systems, and Computers*, pp. 646–650, 2023.
- [16] C. Bitsakos, I. Konstantinou, and N. Koziris, "DERP: A Deep Reinforcement Learning Cloud System for Elastic Resource Provisioning," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2018, pp. 21–29.
- [17] V. Sachidananda and A. Sivaraman, "Erlang: Application-Aware Autoscaling for Cloud Microservices," in *Proc. 19th European Conference on Computer Systems*. New York, NY, USA: Association for Computing Machinery, 2024, pp. 888–923.
- [18] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource Management with Deep Reinforcement Learning," in *Proc. 15th ACM Workshop on Hot Topics in Networks*, 2016, pp. 50–56.
- [19] S. Saxena and K. M. Sivalingam, "DRL-Based Slice Admission Using Overbooking in 5G Networks," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 29–45, 2023.
- [20] Y. Shi, Y. E. Sagduyu, and T. Erpek, "Reinforcement Learning for Dynamic Resource Optimization in 5G Radio Access Network Slicing," in *Proc. IEEE 25th Int. Workshop CAMAD*, 2020, pp. 1–6.
- [21] S. K. Kasi, U. S. Hashmi, S. Ekin, A. Abu-Dayya, and A. Imran, "D-RAN: A DRL-Based Demand-Driven Elastic User-Centric RAN Optimization for 6G & Beyond," *IEEE Trans. Cogn. Commun. Netw.*, vol. 9, no. 1, pp. 130–145, 2023.
- [22] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and Vertical Scaling of Container-Based Applications Using Reinforcement Learning," in *Proc. IEEE 12th CLOUD*, 2019, pp. 329–338.
- [23] S. M. R. Nouri, H. Li, S. Venugopal, W. Guo, M. He, and W. Tian, "Autonomic decentralized elasticity based on a reinforcement learning controller for cloud applications," *Future Generation Computer Systems*, vol. 94, pp. 765–780, 2019.
- [24] C. G. Ralha, A. H. Mendes, L. A. Laranjeira, A. P. Araújo, and A. C. Melo, "Multiagent system for dynamic resource provisioning in cloud computing platforms," *Future Generation Computer Systems*, vol. 94, pp. 80–96, 2019.
- [25] A. Belgacem, S. Mahmoudi, and M. Kihl, "Intelligent multi-agent reinforcement learning model for resources allocation in cloud computing," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 34, no. 6, Part A, pp. 2391–2404, 2022.
- [26] J. Chen, J. Chen, and H. Zhang, "DRLEC: Multi-agent DRL based Elasticity Control for VNF Migration in SDN/NFV Networks," in *Proc. 26th IEEE Asia-Pacific Conf. Commun.*, 2021, pp. 89–93.
- [27] J. Menard, A. Al-Habashna, G. Wainer, and G. Boudreau, "Distributed Resource Allocation In 5G Networks With Multi-Agent Reinforcement Learning," in *Proc. ANNSIM*. IEEE, 2022, pp. 802–813.
- [28] I. Vilà, J. Pérez-Romero, O. Sallent, and A. Umberto, "A Multi-Agent Reinforcement Learning Approach for Capacity Sharing in Multi-Tenant Scenarios," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9450–9465, 2021.
- [29] N. Naderializadeh, J. Sydir, M. Simsek, and H. Nikopour, "Resource Management in Wireless Networks via Multi-Agent Deep Reinforcement Learning," in *Proc. IEEE 21st Int. Workshop Sig. Proc. Adv. Wir. Com.*, 2020, pp. 1–5.
- [30] Google, "Kubernetes," 2014. [Online]. Available: <https://kubernetes.io/>
- [31] C. Ltd., "MicroK8s - Lightweight Kubernetes," 2014. [Online]. Available: <https://microk8s.io/>
- [32] O. Soruce, "Prometheus," 2012. [Online]. Available: <https://prometheus.io/>
- [33] Google, "cAdvisor - Container Advisor," 2014. [Online]. Available: <https://github.com/google/cadvisor>
- [34] G. Labs, "Grafana - Open-Source Observability Platform," 2014. [Online]. Available: <https://grafana.com/>
- [35] Kubernetes, "Horizontal Pod Autoscaler," Documentation, Technical Report, 2015. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [36] —, "Vertical Pod Autoscaler," Documentation, GitHub, Technical Report, 2017. [Online]. Available: <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>
- [37] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with Deep Reinforcement Learning," 2013. [Online]. Available: <https://arxiv.org/abs/1312.5602>
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [39] O. Source, "Kubernetes python client," GitHub repository. [Online]. Available: <https://github.com/kubernetes-client/python>
- [40] I. Docker, "Docker," 2013. [Online]. Available: <https://www.docker.com/>
- [41] J. Prodanov, B. Bertalančić, C. Fortuna, and J. Hribar, "Demonstrating Smart Scaling of AI-Services for Future Networks," in *Proc. IEEE WCNC 2025*, 2025, pp. 1–3.
- [42] K. Rządca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand *et al.*, "Autopilot: Workload Autoscaling at Google Scale," in *Proc. EuroSys 2020*, 2020, pp. 1–16.